Tugas mingguan ke-5

Nama : Silfiana Dwi Nabiilah

NIM : 221344028

Kelas : 2A TNK

Mata Kuliah : Mikroprosesor dan Interface

Dosen : Bapak Ferry Satria, BSEE., M.T.

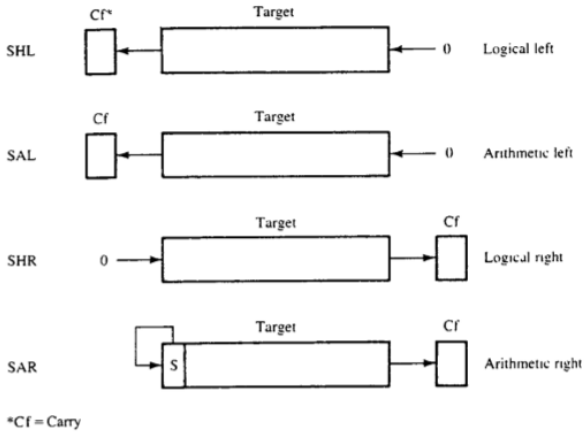
Ibu Rahmawati Hasanah. S.ST., M.T.

A. Intruksi Aritmatika dan Logika

Instruksi shift dan rotasi digunankan untuk memanipulasi bilangan pada tingkat bit biner, seperti yang dilakukan dengan menggunakan instruksi AND, OR, Exclusive-OR, dan NOT. Instruksi shift dan rotate banyak ditemukan pada software aplikasi tingkat rendah untuk mengontrol peralatan I/O. Mikroprosesor memiliki set instruksi lengkap untuk shift dan rotate yang digunakan untuk pergeseran dan pemutaran data pada memori atau register 8 / 16 bit, kecuali register segmen.

1. Penggeseran (Shift)

   Instruksi shift memposisikan atau memindahkan bilangan ke-kanan atau kiri dari register atau lokasi memori. Instruksi ini juga melakukan instruksi aritmetika sederhana seperti perkalian dengan pangkat dari 2+n (geser ke kiri) dan pembagian dengan pangkat 2-n (geser ke kanan). Set instruksi yang dimiliki mikroprosesor berisi 4 instruksi penggeseran yang berbeda; 2 instruksi merupakan penggeseran logika dan 2 instruksi penggeseran aritmetika.



Dibagi menjadi beberapa jenis, yaitu:

1. SHL (Shift Left)
   - Mnemonic : SHL
   - Pengaruh flag : OF, SF, ZF, PF, CF
   - Syntax : SHL Operand,Reg
   - Fungsi : Untuk menggeser bit dari “operand” sebanyak “Reg” kali ke kiri. Bila pergeseran yang dilakukan lebih dari 1 kali maka “Reg” harus berupa regisrer CL atau CX.
   - Contoh:
     MOV BL,00000001b
     SHL BL,1 ; BL=00000010b
     Dengan menggeser n kali kekiri adalah sama bila bilangan tersebut dikali dengan 2 pangkat n.

2. SAL (Arithmatic Shift Left
   - Mnemonic : SAL
   - Syntax : SAL Operand,Reg
   - Pengaruh flag : OF, SF, ZF, PF, CF
   - Fungsi : Untuk menggerser “Operand” sebanyak “Reg” kali ke kiri (sama dengan SHL).

3. SHR (Shift Right)
   - Mnemonic : SHR
   - Syntax : SHR Operand,Reg
   - Pengaruh flag : OF, SF, ZF, PF, CF
   - Fungsi : Untuk menggeser bit dari “operand” sebanyak “Reg” kali ke kanan. Bila pergeseran yang dilakukan lebih dari 1 kali maka “Reg” harus berupa regisrer CL atau CX.
   - Contoh:
     MOV BL,00000100b
     SHR BL,1 ; BL=00000010b
   - Dengan menggeser n kali kekanan adalah sama bila bilangan tersebut dibagi dengan 2 pangkat n.

4. SAR (Arithmatic Shift Right)
   - Mnemonic : SAR
   - Syntax : SAR Operand,Reg
   - Fungsi : Untuk menggeser bit dari “operand” sebanyak “Reg” kali ke kanan. Bit yang keluar dari deretan, masuk ke CF.
   - Sign Bit yang yang dimasukkan ke posisi yang paling kiri mempunyai nilai yang sama seperti sebelum dilakukan pergeseran.
   - Contoh:

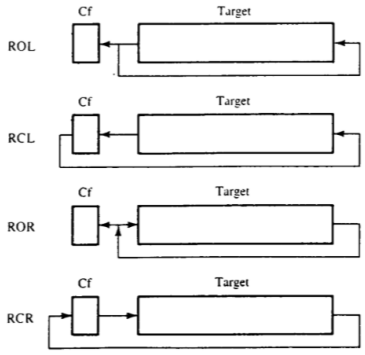
```
MOV AL, 0E0h      ; AL = 11100000b
SAR AL, 1         ; AL = 11110000b,  CF=0.
MOV BL, 4Ch       ; BL = 01001100b
SAR BL, 1         ; BL = 00100110b,  CF=0.
```

Shift kiri aritmatika mempunyai fungsi yang sama dengan shift kiri logic. Namun shift kanan aritmatik berbeda dengan shift kanan logic. **Shift kanan aritmatika** akan selalu membagi **bilangan bertanda** dengan 2 pangkat n (n adalah banyaknya pergeseran); sedangkan shift kanan **logic** akan selalu membagi bilangan tak bertanda dengan 2 pangkat n.

2. Rotate (Pemutaran)

Instruksi rotate menempatkan posisi data biner dengan memutar informasi dalam register atau lokasi memori baik apakah dari satu ujung ke ujung lainnya atau melalui flag carrynya. Terdapat dua tipe instruksi rotasi yaitu rotasi kiri dan rotasi kanan. Mode operasi yang digunakan sama dengan shift. CL digunakan sebagai counter untuk rotasi multiple bit.

Instruksi RCL dan RCR memiliki data yang dirotasi melalui Carry (C) dan ROL dan ROR merotasi hanya melalui target.



Dibagi menjadi beberapa jenis, yaitu:

1) ROL (Rotate Left)
   - Mnemonic : ROL
   - Syntax : ROL Operand,Reg
   - Pengaruh flag : OF, CF
   - Fungsi : Untuk memutar “Operand” sebanyak “Reg” kali ke kiri. Bit yang tergeser keluar dari kiri akan dimasukkan pada CF dan pada bit terkanan dari “Operand”.
   - Pada mikroprosesor 8088 “Reg” haruslah berupa register CL atau CX bila perputaran yang dilakukan terhadap “Operand” lebih dari 1.

     Contoh:

     MOV BL,00100110b ; Misalkan CF=1

     ROL BL,1 ; BL = 01001100b CF=0

2) ROR (Rotate Right)
   - Mnemonic : ROR
   - Syntax : ROR Operand,Reg
   - Pengaruh flag : OF, CF
   - Fungsi : Untuk memutar “Operand” sebanyak “Reg” kali ke kanan. Bit yang tergeser keluar dari kanan akan dimasukkan pada CF dan pada bit terkiri dari “Operand”.
   - Pada mikroprosesor 8088 “Reg” haruslah berupa register CL atau CX bila perputaran yang dilakukan terhadap “Operand” lebih dari 1.

     Contoh:

     MOV BL,00100110b ; Misalkan CF=1

     ROR BL,1 ; BL = 00010011b CF=0

3) RCL (Rotate Through Carry Left)
   - Mnemonic : RCL
   - Syntax : RCL Operand,Reg
   - Pengaruh flag : OF, CF

- Fungsi : Untuk memutar “Operand” sebanyak “Reg” kali ke kiri melalui CF. Bit yang tergeser keluar dari kiri akan dimasukkan pada CF dan nilai CF akan dimasukkan pada bit terkanan dari “Operand”.
- Pada mikroprosesor 8088 “Reg” haruslah berupa register CL atau CX bila perputaran yang dilakukan terhadap “Operand” lebih dari 1.

  Contoh:

  MOV BL,00100110b ; Misalkan CF=1

  RCL BL,1 ; BL = 01001101b CF=0

4) RCR (Rotate Through Carry Right

- Mnemonic : RCR
- Syntax : RCR Operand,Reg
- Pengaruh flag : OF, CF
- Fungsi : Untuk memutar “Operand” sebanyak “Reg” kali ke kanan melalui CF. Bit yang tergeser keluar dari kanan akan dimasukkan pada CF dan nilai CF akan dimasukkan pada bit terkiri dari “Operand”.
- Pada mikroprosesor 8088 “Reg” haruslah berupa register CL atau CX bila perputaran yang dilakukan terhadap “Operand” lebih dari 1.

  Contoh:

  MOV BL,00100110b ; Misalkan CF=1

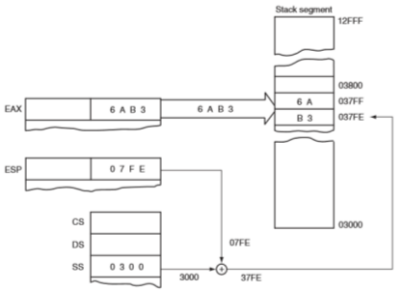
  RCR BL,1 ; BL = 10010011b CF=0

B. Instruksi Pemindahan Data

a) Instruksi PUSH

PUSH dan POP adalah instruksi penting yang digunakan untuk menyimpan dan menarik data dari memori ke stack secara LIFO (Last In First Out). Terdapat empat bentuk instruksi PUSH dan POP: *register, register/memori, segmen register, dan flags. Register addressing* memungkinkan isi semua register 16 bit ditransfer ke atau dari stack.

Instruksi PUSH selalu mentransfer 2 byte data ke dalam stack. Sumber datanya bisa register flag, setiap internal 16-bit, semua register segmen, atau semua 2 bytes memori data. Bila data dimasukkan ke dalam stack, byte data pertama (high byte) disimpan di dalam segmen stack pada lokasi yang dialamatkan oleh SP-1. Byte data kedua (low byte) disimpan di dalam semen stack pada lokasi yang dialamatkan SP-2. Setelah data disimpan di stack, register SP di-dekrementasi oleh 2. Gambar berikut menggambarkan operasi instruksi PUSH AX yang mentransfer isi register AX ke dalam stack.



Hasil dari Instruksi PUSH AX pada register SP dan lokasi memori stack 037FFH dan 037FEH. Sebelum instruksi PUSH AX: nilai SP: 0800. Setelah instruksi PUSH AX: [SP-1]=AH, [SP-2]=AL, dan SP=SP-2=07FE.

Terdapat beberapa jenis PUSH, yaitu:

PUSHF

Instruksi PUSHF meng-copy isi dari register flag pada stack. Seperti instruksi PUSH, PUSHF menyimpan porsi yang paling penting (high byte) dari flag ke dalam stack pada lokasi SP-1 dan porsi yang kurang penting (low byte) pada SP-2. Setelah menyimpan register flag, register SP didekrementasi oleh 2. Opcode dari instruksi PUSH dan PUSHF:

Symbolic	*Byte 1*	*Byte 2*	*Example*
PUSH reg	0101 0rrr		PUSH BX
PUSH mem	1111 1111	mm11 0aaa	PUSH [BX]
PUSH seg	000s s110		PUSH DS
PUSHF	1001 1100		PUSHF

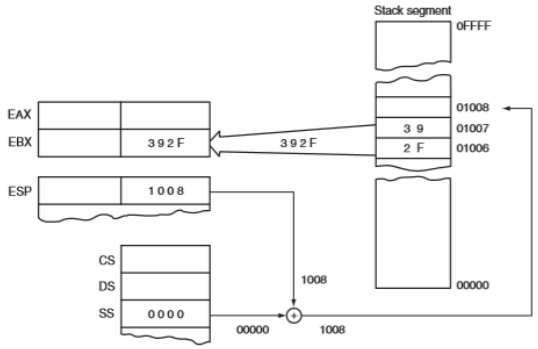
Notes: aaa = any memory addressing mode, mm = MOD code, rr = any 16-bit register, and ss = any segment register.

Syntax Intruksi PUSH:

Symbolic	*Example*	*Note*
PUSH reg16	PUSH BX	16-bit register
PUSH reg32	PUSH EDX	32-bit register
PUSH mem16	PUSH WORD PTR[BX]	16-bit pointer
PUSH mem32	PUSH DWORD PTR[EBX]	32-bit pointer
PUSH mem64	PUSH QWORD PTR[RBX]	64-bit pointer (64-bit mode)
PUSH seg	PUSH DS	Segment register
PUSH imm8	PUSH 'R'	8-bit immediate
PUSH imm16	PUSH 1000H	16-bit immediate
PUSHD imm32	PUSHD 20	32-bit immediate
PUSHA	PUSHA	Save all 16-bit registers
PUSHAD	PUSHAD	Save all 32-bit registers
PUSHF	PUSHF	Save flags
PUSHFD	PUSHFD	Save EFLAGS

b) Intruksi POP

Instruksi POP melakukan operasi inversi dari instruksi PUSH. Instruksi ini memindahkan data dari stack dan menempatkannya ke dalam register, flag atau lokasi memori. Seperti instruksi PUSH, POP tersedia dalam empat bentuk : *register*, *register/memori*, *segmen register*, *dan flags.* Gambar berikut menggambarkan operasi instruksi POP BX. Byte data pertama dipindahkan dari memori stack (yang dialamatkan oleh SP) ke dalam register BL. Byte kedua dipindahkan dari lokasi SP+1 ke register BH. Setelah kedua byte dipindahkan dari stack, pointer stack (SP) diinkrementasi oleh 2.



Hasil dari Instruksi POP BX pada register SP dan lokasi memori stack 01006H dan 01007H. Sebelum dilakukan instruksi POP BX: nilai SP:1006. Setelah dilakukan instruksi POP BX: [SP]=BL, [SP+1]=BH, dan SP=SP+2=1008.

Opcode dan Syntax dari Intruksi POP:

Symbolic	*Byte 1*	*Byte 2*	*Example*
POP reg	0101 1rrr		POP DI
POP mem	1000 1111	mm00 0aaa	POP [DI + 2]
POP seg	000s s111		POP ES
POPF	1001 1101		POPF

Notes: aaa = any memory addressing mode, mm = MOD code, rrr = any 16-bit register, and ss = any segment register.

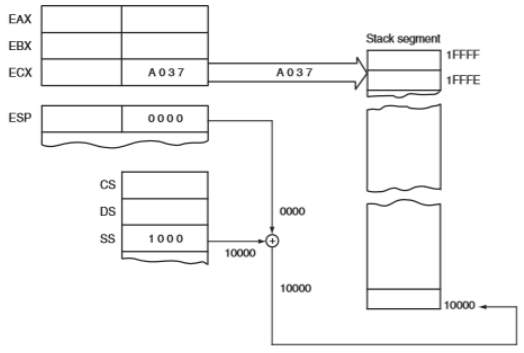
Symbolic	*Example*	*Note*
POP reg16	POP CX	16-bit register
POP reg32	POP EBP	32-bit register
POP mem16	POP WORD PTR[EBX+1]	16-bit pointer
POP mem32	POP DATA3	32-bit memory address
POP mem64	POP FROG	64-bit memory address (64-bit mode)
POP seg	POP FS	Segment register
POPA	POPA	Pops all 16-bit registers
POPAD	POPAD	Pops all 32-bit registers
POPF	POPF	Pops flags
POPFD	POPFD	Pops EFLAGS

c) Instruksi Stack

Stack dapat diartikan tumpukan, yaitu **bagian memori** yang digunakan untuk menyimpan nilai suatu register untuk **sementara**, membentuk tumpukan nilai. *Stack* dapat dibayangkan sebagai tabung memanjang (seperti tabung penyimpan koin). Sedangkan nilai suatu register dapat dibayangkan sebagai koin yang dapat dimasukkan dalam tabung tersebut. Jika ada data yang disimpan maka data-data tersebut akan bergeser ke arah memori rendah, dan akan bergeser kembali ke arah memori tinggi bila data yang disimpan telah diambil.

d) Menginisialisasi Stack

Bila area stack diinisialisasi, baik register segmen stack (SS) maupun pointer stack (SP) dimasukkan. SS sebagai lokasi dasar dari segmen stack. Misal, jika segmen stack menduduki lokasi memori 10000H-1FFFFH, maka register SS diberi 1000H, karena mikroprosesor menggandakan isi dari register segmen dengan 10H untuk mendapatkan memori segmen yang sebenarnya. Untuk memulai stack pada bagian atas segmen, SP diberi alamat 0000H. Gambar berikut menunjukkan bagaimana data masuk ke atas segmen stack dengan instruksi PUSH CX. Ingat, semua segmen bersifat siklus, yaitu lokasi atas sebuah segmen berkaitan dengan lokasi dasar segmen. PUSH CX menggambarkan sifat siklus dari segmen stack. SP dimulai pada 0000H dan setelah didekrementasi, berakhir pada FFFEH.



e) Instruksi LAHF dan SAHF

LAHF (Load Flags Into AH Register) adalah instruksi yang berfungsi untuk mengcopykan 8 low bits dari flag register CF, PF, AF, ZF dan SF yang terletak pada bit ke 0, 2, 4, 6 dan 7 dari flags register menuju register AH pada bit yang sesuai.

7	6	5	4	3	2	1	0
[SF]	[ZF]	[0]	[AF]	[0]	[PF]	[1]	[CF]

note: bit 1,3, dan 5 telah ditetapkan.

SAHF (Load Flags Into AH Register) adalah instruksi yang berfungsi untuk mengirimkan data pada register AH ke dalam 8 low bits dari flag register CF, PF, AF, ZF dan SF yang terletak pada bit ke 0, 2, 4, 6.

7	6	5	4	3	2	1	0
[SF]	[ZF]	[0]	[AF]	[0]	[PF]	[1]	[CF]

note: bit 1,3, dan 5 telah ditetapkan.

f) Instruksi TEST

Instruksi Test berfungsi untuk melakukan operasi AND antara "Operand1" dan "Operand2". Hasil dari operasi AND hanya mempengaruhi flag register saja dan tidak mempengaruhi nilai "operand1" maupun "operand2". Adapun flag register yang terpengaruh instruksi test adalah OF, SF, ZF, PF, dan CF. Syntax : TEST Operand1,Operand2

Contoh :

```
MOV AL,000001010b
TEST AL, 1      ;ZF=0
TEST AL, 10b    ;ZF=1
RET
```

C. Hal-Hal yang tidak dapat dilakukan dalam pemindahan data

1. Pemindahan data antar register segmen (ds, es, cs, ss)

   mov ds, es → tidak dibenarkan

solusi : gunakan register general, misalnya register ax, sebagai perantara:

```
mov ax, es
mov ds, ax
```

atau gunakan *stack* sebagai perantara:

```
push es
pop ds
```

2. Pemindahan data secara langsung untuk register segmen (ds, es, cs, ss)

   mov ds, 123H → tidak dibenarkan

Solusi : gunakan register general, misalnya register ax, sebagai perantara.

```
mov ax, 123H
mov ds, ax
```

3. Pemindahan data langsung antar memori

   mov memB, memA → tidak dibenarkan

Solusi : Gunakan register general, misalnya register ax, sebagai perantara.

mov ax, memA

mov memB, ax

4. Pemindahan data antar register general yang berbeda daya tampungnya (8 bit dengan 16 bit) tanpa pointer).

   mov al, bx →tidak dibenarkan.

D. Operasi Pengurangan

Pada operasi pengurangan ini digunakan beberapa jenis perintah, diantaranya yaitu sebagai berikut:

1) SUB

Perintah SUB akan mengurangkan nilai pada Tujuan dengan Asal. Hasil yang didapat akan ditaruh pada Tujuan, dalam bahasa pascal sama dengan instruksi **Tujuan:=Tujuan-Asal**. Unruk menolkan suatu register bisa menggunakan perintah seperti ini : SUB AX, AX.

Syntak yang digunakan: **SUB Tujuan, Asal**

2) SBB

Seperti pada operasi penambahan, maka pada operasi pengurangan dengan bilangan yang besar(lebih dari 16 bit), bisa anda gunakan perintah SUB disertai dengan SBB(Substract With Carry). Perintah SBB digunakan dengan syntax: **SBB Tujuan, Asal**

Perintah SBB akan mengurangkan nilai Tujuan dengan Asal dengan cara yang sama seperti perintah SUB, kemudian hasil yang didapat dikurangi lagi dengan Carry Flag(Tujuan:=Tujuan-Asal-CF).

3) DEC

Perintah DEC(Decrement) digunakan khusus untuk pengurangan dengan 1. Perintah DEC hanya menggunakan 1 byte memory, sedangkan perintah SUB dan SBB menggunakan 3 byte. Oleh sebab itu bila anda ingin melakukan operasi pengurangan dengan 1 gunakanlah perintah DEC. Syntax pemakaian perintah dec ini adalah: **DEC Tujuan**

Nilai pada tujuan akan dikurangi 1, seperti perintah Tujuan:=Tujuan-1 dalam Turbo Pascal. Tujuan disini dapat berupa suatu register maupun memory. Contoh : perintah DEC AL akan mengurangi nilai di register AL dengan 1.